

HTML SVG

A Practical, From-Zero Learning Guide

Part 12 — Complete SVG Projects & Practice

This part changes the learning style: instead of studying isolated SVG features, you will build complete projects and combine shapes, paths, gradients, transforms, animation, JavaScript and React.

Goal: Learn how to think through an SVG project from idea → coordinate system → geometry → styling → interaction → responsive behavior → accessibility.

1. How to Approach an SVG Project

Do not start by writing random path commands. Break the design into layers and components.

- Define the final size and viewBox.
- Divide the illustration into major groups.
- Build the simplest geometry first.
- Add colors and strokes.
- Add reusable definitions such as gradients or filters.
- Add interaction or animation only after the static version works.
- Make the SVG responsive.
- Add accessibility.
- Optimize only after the design is correct.

2. Project 1 — Animated Check Icon

A small project for practicing paths, stroke animation and CSS.

```
<svg class="check-icon"
      viewBox="0 0 100 100"
      aria-label="Success">

  <circle class="ring"
        cx="50" cy="50" r="42"/>

  <path class="check"
        d="M28 52 L43 67 L73 35"/>
</svg>

<style>
.check-icon {
  width: 100px;
}

.ring,
.check {
  fill: none;
  stroke: currentColor;
  stroke-width: 7;
  stroke-linecap: round;
  stroke-linejoin: round;
}

.ring {
  stroke-dasharray: 264;
  stroke-dashoffset: 264;
  animation: draw .7s ease forwards;
}

.check {
  stroke-dasharray: 70;
  stroke-dashoffset: 70;
  animation: draw .45s .65s ease forwards;
}

@keyframes draw {
  to { stroke-dashoffset: 0; }
}
</style>
```

The important idea is to obtain approximate path lengths, set dasharray/dashoffset, and animate dashoffset to zero.

3. Project 1 Improvements

- Add a reduced-motion media query.
- Change the icon color through currentColor.
- Turn it into a React component.

- Add a success/error variant.
- Allow the size to be controlled with a prop.

4. Project 2 — Animated Hamburger Menu

Build a three-line menu icon that transforms into an X.

```
<button class="menu-button"
  aria-label="Toggle menu">
  <svg viewBox="0 0 100 100">
    <line class="top"
      x1="20" y1="30"
      x2="80" y2="30"/>
    <line class="middle"
      x1="20" y1="50"
      x2="80" y2="50"/>
    <line class="bottom"
      x1="20" y1="70"
      x2="80" y2="70"/>
  </svg>
</button>

<style>
.menu-button line {
  fill: none;
  stroke: currentColor;
  stroke-width: 8;
  stroke-linecap: round;
  transition: transform .25s,
    opacity .25s;
  transform-box: fill-box;
}

.menu-button.open .top {
  transform: translateY(20px) rotate(45deg);
}

.menu-button.open .middle {
  opacity: 0;
}

.menu-button.open .bottom {
  transform: translateY(-20px) rotate(-45deg);
}
</style>
```

The main lesson is that an SVG icon can be treated like any other interactive UI component.

5. Project 3 — Circular Progress Ring

```
<svg class="progress"
  viewBox="0 0 120 120">
  <circle
    cx="60" cy="60" r="45"
    class="track"/>

  <circle
    cx="60" cy="60" r="45"
    class="value"
    pathLength="100"
    stroke-dasharray="72 28"
    transform="rotate(-90 60 60)"/>

  <text x="60" y="67"
    text-anchor="middle">
    72%
  </text>
</svg>
```

Using `pathLength="100"` makes the stroke behave like a percentage scale. A `dasharray` of 72 28 represents 72% visible stroke followed by 28% gap.

6. Project 3 — Dynamic Progress

```
function ProgressRing({ value }) {
  return (
    <svg viewBox="0 0 120 120">
      <circle
        cx="60" cy="60" r="45"
```

```

        fill="none"
        stroke="lightgray"
        strokeWidth="10"
      />

      <circle
        cx="60" cy="60" r="45"
        fill="none"
        stroke="currentColor"
        strokeWidth="10"
        pathLength="100"
        strokeDasharray={` ${value} ${100 - value}`}
        transform="rotate(-90 60 60)"
      />

      <text
        x="60" y="67"
        textAnchor="middle"
      >
        {value}%
      </text>
    </svg>
  );
}

```

7. Project 4 — Five-Star Rating

Instead of drawing every star manually, define one reusable star and use it repeatedly.

```

<svg viewBox="0 0 500 100">
  <defs>
    <path id="star"
      d="M50 5
        L61 38 L95 38
        L68 58 L78 92
        L50 72 L22 92
        L32 58 L5 38
        L39 38 Z"/>
    </defs>

    <use href="#star"
      transform="translate(0 0)"
      fill="gold"/>

    <use href="#star"
      transform="translate(100 0)"
      fill="gold"/>

    <use href="#star"
      transform="translate(200 0)"
      fill="gold"/>

    <use href="#star"
      transform="translate(300 0)"
      fill="lightgray"/>

    <use href="#star"
      transform="translate(400 0)"
      fill="lightgray"/>
  </svg>

```

This project reinforces , reusable paths, transforms and component-like thinking.

8. Project 5 — Interactive Star Rating

```

function Rating({ value, onChange }) {
  return (
    <div>
      {[1, 2, 3, 4, 5].map(star => (
        <button
          key={star}
          onClick={() => onChange(star)}
          aria-label={` ${star} stars`}
        >
          <svg
            viewBox="0 0 100 100"
            width="32"
            height="32"
            fill={star <= value
              ? "gold"
              : "lightgray"}
          >

```

```

        <path d="M50 5 L61 38 L95 38
              L68 58 L78 92
              L50 72 L22 92
              L32 58 L5 38
              L39 38 Z"/>
      </svg>
    </button>
  )}}
</div>
);
}

```

Putting the SVG inside a real button gives the control better keyboard semantics than making a raw SVG pretend to be a button.

9. Project 6 — Animated Food Delivery Route

```

<svg viewBox="0 0 600 300">
  <path id="route"
    d="M40 230
      C100 50 190 50 280 170
      S450 260 560 60"
    fill="none"
    stroke="currentColor"
    stroke-width="5"
    stroke-dasharray="12 10"/>

  <circle r="12"
    fill="tomato">
    <animateMotion
      dur="6s"
      repeatCount="indefinite">
      <mpath href="#route"/>
    </animateMotion>
  </circle>

  <circle cx="40" cy="230"
    r="10" fill="seagreen"/>

  <circle cx="560" cy="60"
    r="10" fill="royalblue"/>
</svg>

```

This resembles a simplified food-delivery route: restaurant, moving delivery marker and customer destination.

10. Project 6 Improvements

- Add a restaurant icon at the start.
- Add a customer icon at the destination.
- Show a progress percentage.
- Animate only when the order is active.
- Pause animation when the order is cancelled.
- Add a reduced-motion alternative.
- Make the route data-driven in React.

11. Project 7 — SVG Weather Card

```

<svg viewBox="0 0 300 180">
  <circle cx="75" cy="65"
    r="28"
    fill="gold"/>

  <g stroke="gold"
    stroke-width="5"
    stroke-linecap="round">
    <line x1="75" y1="20"
      x2="75" y2="8"/>
    <line x1="75" y1="110"
      x2="75" y2="122"/>
    <line x1="30" y1="65"
      x2="18" y2="65"/>
    <line x1="120" y1="65"

```

```

        x2="132" y2="65"/>
</g>

<path
  d="M120 120
    C110 90 135 75 155 90
    C165 60 210 65 215 100
    C245 95 260 125 245 145
    H125
    C105 145 105 120 120 120 Z"
  fill="lightgray"/>
</svg>

```

Practice combining circles, groups, lines and a custom cloud path.

12. Project 8 — Animated Logo

```

<svg class="logo"
  viewBox="0 0 300 100">
  <g class="mark">
    <circle cx="50" cy="50" r="35"/>
    <path d="M30 52 L50 30 L70 52
      L50 70 Z"/>
  </g>

  <text x="100" y="62"
    font-size="36"
    font-weight="700">
    CraveCart
  </text>
</svg>

<style>
.mark {
  transform-origin: 50px 50px;
  animation: pop .7s ease-out;
}

@keyframes pop {
  from {
    transform: scale(.2);
    opacity: 0;
  }
  to {
    transform: scale(1);
    opacity: 1;
  }
}
</style>

```

For a real logo, build the mark from deliberate geometry rather than the placeholder shapes above.

13. Project 9 — Interactive Bar Chart

```

function BarChart({ values }) {
  const max = Math.max(...values);

  return (
    <svg viewBox="0 0 500 250">
      {values.map((value, i) => {
        const height =
          (value / max) * 180;

        return (
          <rect
            key={i}
            x={30 + i * 60}
            y={210 - height}
            width="35"
            height={height}
            rx="5"
          />
        );
      })}
    </svg>
  );
}

```

Important: chart data and visual geometry should be separate concepts. First calculate scales and coordinates, then render SVG.

14. Project 9 — Hover Tooltip

```
function Point({ x, y, label }) {
  const [show, setShow] = useState(false);

  return (
    <g
      onMouseEnter={() => setShow(true)}
      onMouseLeave={() => setShow(false)}
    >
      <circle
        cx={x}
        cy={y}
        r="6"
        tabIndex="0"
      />

      {show && (
        <text x={x + 10} y={y - 10}>
          {label}
        </text>
      )}
    </g>
  );
}
```

For keyboard users, extend this pattern with focus handling and an accessible textual equivalent.

15. Project 10 — Interactive Restaurant Map

SVG is excellent for schematic maps where exact geographic mapping is not required.

```
<svg viewBox="0 0 600 400">
  <rect width="600" height="400"
    fill="#f3f3f3"/>

  <path
    d="M40 80 H560
      M40 200 H560
      M40 320 H560
      M120 30 V370
      M300 30 V370
      M470 30 V370"
    stroke="white"
    stroke-width="28"/>

  <g class="restaurant"
    tabindex="0">
    <circle cx="120" cy="120"
      r="18"/>
    <text x="145" y="125">
      Restaurant A
    </text>
  </g>

  <g class="restaurant"
    tabindex="0">
    <circle cx="360" cy="240"
      r="18"/>
    <text x="385" y="245">
      Restaurant B
    </text>
  </g>
</svg>
```

Make each restaurant a reusable component and attach click/focus behavior.

16. Project 10 — Selected Restaurant

```
function RestaurantMarker({
  x, y, name, selected, onClick
}) {
  return (
    <g
      role="button"
      tabIndex={0}
      onClick={onClick}
      className={selected
        ? "selected"
        : ""}
    >
      <circle
```

```

        cx={x}
        cy={y}
        r={selected ? 22 : 16}
      />
      <text
        x={x + 28}
        y={y + 5}
      >
        {name}
      </text>
    </g>
  );
}

```

A production implementation should also handle Enter/Space keyboard activation and communicate selected state with an appropriate ARIA attribute.

17. Project 11 — Donut Chart

A donut chart can be built from circles using `stroke-dasharray` and `stroke-dashoffset`.

```

<svg viewBox="0 0 120 120">
  <circle
    cx="60" cy="60" r="42"
    fill="none"
    stroke="#ddd"
    stroke-width="16"/>

  <circle
    cx="60" cy="60" r="42"
    fill="none"
    stroke="tomato"
    stroke-width="16"
    pathLength="100"
    stroke-dasharray="65 35"
    transform="rotate(-90 60 60)"/>

  <text x="60" y="66"
    text-anchor="middle">
    65%
  </text>
</svg>

```

For multiple categories, calculate each segment's percentage and cumulative dash offset.

18. Project 12 — Animated Order Timeline

```

<svg viewBox="0 0 700 140">
  <line
    x1="80" y1="60"
    x2="620" y2="60"
    stroke="lightgray"
    stroke-width="8"/>

  <line
    x1="80" y1="60"
    x2="360" y2="60"
    stroke="seagreen"
    stroke-width="8"
    stroke-dasharray="280"
    stroke-dashoffset="280">
    <animate
      attributeName="stroke-dashoffset"
      from="280"
      to="0"
      dur="1.5s"
      fill="freeze"/>
  </line>

  <circle cx="80" cy="60" r="16"/>
  <circle cx="360" cy="60" r="16"/>
  <circle cx="620" cy="60" r="16"/>
</svg>

```

This pattern is useful for order placed → confirmed → prepared → delivered timelines.

19. Project 13 — SVG Loading Spinner

```

<svg viewBox="0 0 100 100"

```



```

    class="spinner"
    aria-label="Loading">
<circle
  cx="50" cy="50" r="35"
  fill="none"
  stroke="currentColor"
  stroke-width="8"
  stroke-linecap="round"
  stroke-dasharray="80 140"/>
</svg>

<style>
.spinner {
  animation: spin 1s linear infinite;
}

@keyframes spin {
  to {
    transform: rotate(360deg);
  }
}

@media (prefers-reduced-motion: reduce) {
  .spinner {
    animation: none;
  }
}
</style>

```

20. Project 14 — Empty Cart Illustration

Combine simple geometry to create an illustration rather than using a raster image.

```

<svg viewBox="0 0 400 300"
  role="img"
  aria-labelledby="title">
<title id="title">
  Empty shopping cart
</title>

<path
  d="M70 70 H105
    L130 190
    H300
    L330 100
    H115"
  fill="none"
  stroke="currentColor"
  stroke-width="12"
  stroke-linejoin="round"/>

<circle cx="155" cy="225"
  r="16"/>
<circle cx="275" cy="225"
  r="16"/>

<path
  d="M155 140
    C175 110 210 110 230 140
    C250 110 285 110 305 140"
  fill="none"
  stroke="currentColor"
  stroke-width="8"/>
</svg>

```

This is a good exercise in building an illustration from primitives and paths.

21. Project 15 — Complete React Mini Project

Build a reusable order-status component with SVG, React state and CSS.

```

import { useState } from "react";

const steps = [
  "Placed",
  "Confirmed",
  "Preparing",
  "Delivered"
];

export default function OrderTracker() {

```

```

const [active, setActive] = useState(1);

return (
  <div>
    <svg
      viewBox="0 0 800 160"
      className="w-full"
      role="img"
      aria-label={`Order status:
        ${steps[active]}`}
    >
      <line
        x1="80" y1="60"
        x2="720" y2="60"
        stroke="lightgray"
        strokeWidth="8"
      />

      <line
        x1="80" y1="60"
        x2={80 + active * 213}
        y2="60"
        stroke="seagreen"
        strokeWidth="8"
      />

      {steps.map((step, i) => (
        <g key={step}>
          <circle
            cx={80 + i * 213}
            cy="60"
            r="22"
            fill={i <= active
              ? "seagreen"
              : "lightgray"}
          />

          <text
            x={80 + i * 213}
            y="110"
            textAnchor="middle"
          >
            {step}
          </text>
        </g>
      )))}
    </svg>

    <div>
      {steps.map((step, i) => (
        <button
          key={step}
          onClick={() => setActive(i)}
        >
          {step}
        </button>
      )))}
    </div>
  </div>
);
}

```

This project combines React state, SVG geometry, dynamic attributes, accessibility and responsive sizing.

22. Project Architecture

```

src/
  components/
    svg/
      CheckIcon.jsx
      SearchIcon.jsx
      Rating.jsx
      ProgressRing.jsx
      OrderTracker.jsx
      DeliveryRoute.jsx
      Chart.jsx

  pages/
    Dashboard.jsx
    OrderDetails.jsx

  data/
    chartData.js
    orderSteps.js

```

Keeping reusable SVG components in a dedicated area can make a larger React project easier to maintain.

23. SVG Project Workflow

- 1. Sketch the design.
- 2. Establish the viewBox.
- 3. Draw the major shapes.
- 4. Group related elements.
- 5. Add reusable definitions.
- 6. Add colors and typography.
- 7. Add responsive behavior.
- 8. Add interaction.
- 9. Add animation.
- 10. Add accessibility.
- 11. Test.
- 12. Optimize.

24. When NOT to Use SVG

- Use raster images when the source is photographic.
- Do not turn a huge photograph into thousands of SVG paths.
- Do not use SVG just because it is fashionable.
- For simple text, use HTML text.
- For decorative backgrounds, CSS may be simpler.
- For large geographic maps, specialized mapping libraries may be more appropriate.

25. Project Practice Levels

Beginner: check icon, hamburger, spinner, star rating.

Intermediate: progress ring, weather card, empty-cart illustration, order timeline.

Advanced: delivery route, interactive chart, restaurant map, donut chart.

Professional: complete React SVG component system with accessibility, responsive behavior, reusable definitions and optimized assets.

26. Practice Challenge A — Without Looking

- Create a 100×100 SVG.
- Draw a circle in the center.
- Add a line through it.
- Rotate the line around the center.
- Animate the rotation.
- Make the color follow `currentColor`.

27. Practice Challenge B — From Data

- Create an array of seven order counts.
- Find the maximum.
- Scale the values to 180 SVG units.
- Render bars using `map()`.
- Add labels.
- Highlight the largest bar.
- Add a tooltip.

28. Practice Challenge C — CraveCart

- Create a restaurant marker component.
- Show restaurant name.
- Add a selected state.
- Animate the selected marker.
- Create a route to the customer.
- Animate a delivery marker along the route.
- Add order status.
- Make the entire visualization responsive.

29. Debugging Projects

- If nothing appears: inspect `viewBox` and dimensions.
- If a shape is misplaced: inspect coordinate calculations.
- If rotation is wrong: inspect transform origin and transform order.
- If a gradient is missing: inspect its ID and `url(#...)`.
- If a mask/filter is clipped: inspect the filter/mask region.
- If React components interfere: check duplicate IDs.
- If animation is invisible: inspect `dasharray`/`dashoffset` and animation timing.

30. Final Project — CraveCart Delivery Visualization

Build a single React component that contains:

- A simplified city/road background.
- Restaurant marker.
- Customer marker.
- Curved delivery route.
- Animated delivery rider marker.
- Order-status timeline.
- Progress percentage.
- Accessible description.
- Reduced-motion behavior.
- Responsive `viewBox`.

- Reusable React subcomponents.

31. Suggested Component Tree

```
<DeliveryDashboard>
  <DeliveryMap>
    <Roads />
    <RestaurantMarker />
    <CustomerMarker />
    <DeliveryRoute />
    <DeliveryMarker />
  </DeliveryMap>

  <OrderStatus />
  <ProgressRing />
</DeliveryDashboard>
```

32. What This Project Teaches

- SVG composition
- viewBox
- groups
- paths
- markers
- animation
- React state
- props
- data-driven rendering
- accessibility
- responsive design
- component architecture

33. Portfolio Advice

Do not publish only screenshots. For an SVG project, show the source code, explain the SVG concepts used, show an interactive demo and describe one technical challenge you solved.

- Show the final result.
- Explain the coordinate system.
- Explain one interesting path or transform.
- Explain how animation works.
- Explain accessibility decisions.
- Explain how the React component is reusable.

34. Part 12 Master Checklist

- Can you build an SVG without copying an entire file?
- Can you choose a useful viewBox?
- Can you divide an illustration into groups?
- Can you reuse paths with defs/use?
- Can you animate strokes?

- Can you animate objects along paths?
- Can you create responsive SVG charts?
- Can you connect SVG to React state?
- Can you make interactive SVG controls accessible?
- Can you debug coordinate and ID problems?
- Can you turn a design into reusable SVG components?

35. Final Challenge

Build a complete **CraveCart Order Tracking Illustration** without following a tutorial. It should contain a restaurant, customer, road, route, delivery marker, order timeline and progress ring. Make it responsive and accessible. Then rebuild the same project as reusable React components.

If you can complete this independently, you have moved from learning SVG syntax to actually **using SVG as a development skill**.

36. Next Part

Part 13 will focus on **SVG + React Advanced Real-World Projects**: reusable icon systems, component APIs, dynamic charts, SVG state management, advanced interactions, React animation strategies and a larger application-style SVG project.